

Contents

1	DD12: 利用量計測・課金	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	2
1.4.1	3.1 リミット設計（警告 / 拒否 2 段階閾値）	2
1.4.2	3.2 グレースフルデグレード	2
1.4.3	3.3 データ保持期間	2
1.4.4	3.4 月次集計（UTC 15:00、月末日のみ実行）	2
1.4.5	3.4.1 ダッシュボード最終更新タイムスタンプ取得元（FR-135d / FR-135g）	3
1.4.6	3.4.2 取得失敗時のフォールバック表示（FR-135g）	3
1.4.7	3.5 月次確定（JST 02:00、月初 1 日）	3
1.4.8	3.6 Stripe Webhook（IF #10 経由）	4
1.4.9	3.7 PCI-DSS スコープ外宣言	5
1.4.10	3.8 80% 警告 / 100% / 125% アラート	5
1.4.11	3.9 関連する横断設計	5
1.5	4. 関連設計	5
1.6	5. テスト観点	5
1.6.1	5.1 その他観点	5
1.7	6. 未確定事項・確認事項	5

1 DD12: 利用量計測・課金

1.1 0. 文書情報

項目	内容
文書名	DD12: 利用量計測・課金
詳細設計 ID	DD12
対象システム	FAQ AI ウィジェット SaaS / メインシステム
関連機能 ID	FR-120～127（利用量・課金ダッシュボード） / FR-139（Stripe Webhook 経由） / AC-020 / AC-040
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
機能	FR-120～127	利用量・課金ダッシュボード / 月次集計 / 80% 警告 / 125% 拒否
機能	FR-139	Stripe 9 ハンドラ（IF #10 経由）
画面	SCR-015	利用量・課金ダッシュボード
API	/usage /billing/*	利用量取得 / 請求書
Webhook	/webhooks/stripe	Stripe 9 イベント受信（IF #10）
テーブル	usage_metering billing_invoices question_logs	利用量・課金基盤
cron	monthlyAggregate / monthlyFinalize	月次集計・確定

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 13.7	データ保持・削除	question_logs 1 年 / billing 7 年
§ 13.8	リミット設計	警告 / 拒否の 2 段階閾値（FAQ / プロジェクト / 月次質問数 / 質問レート）
§ 14.1.1	月次集計（UTC 15:00、月末日のみ実行）	usage_metering 集計

元章	元タイトル	概要
§ 14.1.2	月次確定 (JST 02:00、月初 1 日)	billing_invoices 発行 + IF #10
§ 6 SCR-015	画面詳細設計	利用量・課金ダッシュボード (正本は基本設計)
§ 7	機能詳細設計 (課金関連)	プラン / 課金 (正本は基本設計)

1.4 3. 詳細設計本文

1.4.1 3.1 リミット設計 (警告 / 拒否 2 段階閾値)

項目	警告閾値	拒否閾値	拒否時の応答
FAQ 件数 / 契約	8,000	12,000	409 FAQ_LIMIT_EXCEEDED
プロジェクト数 / 契約	40	50	409 PROJECT_LIMIT_EXCEEDED
月次質問数	80% 無料枠	125% 無料枠	通常は事後課金、超過は受信レート 1/2
ウィジェット質問レート	50/min	60/min	429 RATE_LIMITED (Retry-After 付)
ウィジェット質問同時	-	オーナー 10 / グローバル 200	429 RATE_LIMITED

1.4.2 3.2 グレースフルデグレード

```
// 503 + Retry-After
```

```
c.status(503);
```

```
c.header('Retry-After', '60');
```

```
return c.json({ code: 'SERVICE_UNAVAILABLE', detail: '一時的に利用できません。1 分後に再度お試しください。' });
```

NFR-114 / 117 に基づき、AI 推論不可時もウィジェットは「管理者ユーザー誘導」モードで応答継続。

1.4.3 3.3 データ保持期間

データ種別	保持期間	起点	削除方式
question_logs	1 年	created_at	物理削除 cron
監査ログ (billing)	7 年	created_at	tombstone (本文物理削除、ハッシュは残す)
billing_invoices	7 年 (電子帳簿保存法)	issued_at	論理削除

1.4.4 3.4 月次集計 (UTC 15:00、月末日のみ実行)

```
// app/workers/cron/src/jobs/monthly-aggregate.ts
```

```
// 実行条件: UTC 15:00 かつ DATE_ADD(date, '+1 day') 月が変わる場合
```

```
export async function monthlyAggregate(env: Env) {
```

```
  // オーナー一覧 = contract_owners 行 ('valid=1' かつ 'contract_status != 'deleted')。
```

```
  // user_id が旧 accounts.id と物理値同じ ULID。
```

```
  const owners = await env.DB.prepare(`
```

```
    SELECT user_id FROM contract_owners
```

```
    WHERE valid = 1 AND contract_status != 'deleted'
```

```
`).all<{ user_id: string }>();
```

```
  const ym = currentBillingYearMonth(); // 'YYYY-MM'
```

```
  for (const t of owners.results) {
```

```
    const usage = await env.DB.prepare(`
```

```
      SELECT
```

```
        COUNT(*) FILTER (WHERE metering_billable = 1) AS question_count,
```

```
        COALESCE(SUM(ai_token_count_input), 0) AS token_input,
```

```
        COALESCE(SUM(ai_token_count_output), 0) AS token_output
```

```
      FROM question_logs WHERE contract_owner_user_id = ?1
```

```
      AND created_at >= ?2 AND created_at < ?3
```

```
`).bind(t.user_id, monthStart(ym), monthEnd(ym)).first();
```

```

const faqSnapshot = await env.DB.prepare(`
  SELECT COUNT(*) c FROM faqs
  WHERE project_id IN (SELECT id FROM projects WHERE contract_owner_user_id = ?1)
  AND status = 'published'
`).bind(t.user_id).first<{ c: number }>();

const chatRoomCount = await env.DB.prepare(`
  SELECT COUNT(*) c FROM chat_rooms cr
  JOIN inquiries i ON i.id = cr.inquiry_id
  WHERE i.contract_owner_user_id = ?1 AND cr.created_at >= ?2 AND cr.created_at < ?3
`).bind(t.user_id, monthStart(ym), monthEnd(ym)).first<{ c: number }>();

await env.DB.prepare(`
  INSERT OR REPLACE INTO usage_metering (
    id, contract_owner_user_id, billing_year_month,
    question_count, faq_count_snapshot, chat_room_count,
    ai_token_input, ai_token_output, ai_cost_yen,
    updated_at
  ) VALUES (?1, ?2, ?3, ?4, ?5, ?6, ?7, ?8, ?9, ?10)
`).bind(
  generateUuid(), t.user_id, ym,
  usage.question_count, faqSnapshot?.c ?? 0, chatRoomCount?.c ?? 0,
  usage.token_input, usage.token_output,
  computeAiCost(usage.token_input, usage.token_output),
  new Date().toISOString(),
).run();
}
}

```

1.4.5 3.4.1 ダッシュボード最終更新タイムスタンプ取得元 (FR-135d / FR-135g)

SCR-015 / SCR-026 の KPI カード群右上に「最終更新: YYYY-MM-DD HH:MM(JST)」を表示する(基本設計 01_画面設計.md §1.5.1、メッセージ 06_メッセージ一覧.md MSG-SCR-015-LBL-001 / MSG-SCR-026-LBL-007)。

- 取得元の優先順位:
 1. `usage_metering.updated_at` の最大値(当該オーナースコープ、当月 `billing_year_month`)
 2. ない場合は当月の `question_logs.created_at` の最大値(リアルタイム代替)
 3. それも 0 件のときは `null`(画面側で「—」+ Tooltip「集計予定」表示)
- 集計遅延判定: 取得した最終更新時刻が現在時刻から 5 分以上前のとき、画面側で「☒ 集計遅延(最終: …)」を黄表記で警告(MSG-SCR-015-LBL-002 / MSG-SCR-026-LBL-008)。閾値 5 分はリアルタイム代替経由のときも適用。
- API レスポンスメタ: GET `/usage` のレスポンスに `meta.last_aggregated_at`(ISO 8601、JST) を含める。

1.4.6 3.4.2 取得失敗時のフォールバック表示 (FR-135g)

KPI / 一覧の取得に失敗した場合(API 5xx / タイムアウト / ネットワーク断)、画面は「取得に失敗しました(再読込)」+ 赤縁(MSG-SCR-015-MSG-FETCH-001 / MSG-SCR-026-MSG-FETCH-001)を表示し、Tooltip に該当 E-* コードを示す。再読込ボタンのみ活性化し、それ以外のクリック導線は非活性化する。

- 再試行ロジック: クライアント側で指数バックオフ(初回 1s → 最大 30s、最大 3 回)を行ったうえで失敗確定。
- 取得失敗中のリンク: 該当 KPI / 一覧行の他要素(KPI クリック、ドリルダウン)はすべて `aria-disabled="true"` + カーソル `not-allowed`。
- エラーコード対応: API 側は E-* を返す(E-SYS-UNAVAILABLE 系)。具体のコードは 05_エラー設計.md を参照。

1.4.7 3.5 月次確定 (JST 02:00、月初 1 日)

```

// app/workers/cron/src/jobs/monthly-finalize.ts
export async function monthlyFinalize(env: Env) {
  const prevYm = previousBillingYearMonth();
  // 課金対象オーナー = contract_owners 行 ('valid=1' かつ 'contract_status='active'` かつ `contract

```

```

const owners = await env.DB.prepare(`
  SELECT user_id AS id, contract_plan AS plan FROM contract_owners
  WHERE valid = 1 AND contract_status = 'active' AND contract_plan != 'free'
`).all<TenantRow>();

for (const t of owners.results) {
  const usage = await env.DB.prepare(`
    SELECT * FROM usage_metering WHERE contract_owner_user_id = ?1 AND billing_year_month = ?2
  `).bind(t.id, prevYm).first<UsageRow>();
  if (!usage) continue;
  // idempotent key で重複生成抑制
  const idemKey = `invoice_finalize:${t.id}:${prevYm}`;
  const dup = await env.KV_DEDUP.get(idemKey);
  if (dup) continue;

  const amount = calculateInvoiceAmount(t.plan, usage);
  const invoiceId = generateUlid();
  await env.DB.prepare(`
    INSERT INTO billing_invoices
    (id, contract_owner_user_id, billing_year_month, status, amount_total, amount_tax, issued_at)
    VALUES (?1, ?2, ?3, 'issued', ?4, ?5, ?6)
  `).bind(invoiceId, t.id, prevYm, amount.total, amount.tax,
    new Date().toISOString()).run();

  await env.DB.prepare(`UPDATE usage_metering SET finalized_at = ?1 WHERE id = ?2`)
    .bind(new Date().toISOString(), usage.id).run();

  // IF #10 経由で Stripe へ請求書発行依頼 (顧客経由)
  await sendToAdminIntegration(env, '/internal/main-integration/v1/billing/invoice-issued', {
    invoiceId, contractOwnerUserId: t.id, ym: prevYm, amount: amount.total,
  });

  await enqueueNotification(env, {
    contractOwnerUserId: t.id, kind: 'BILLING_INVOICE_ISSUED', refId: invoiceId,
  });

  await env.KV_DEDUP.put(idemKey, '1', { expirationTtl: 90 * 86400 });
  await writeAudit(env, {
    action: 'billing.invoice.issue', contractOwnerUserId: t.id, targetId: invoiceId,
    retentionClass: 'billing',
  });
}
}

```

1.4.8 3.6 Stripe Webhook (IF #10 経由)

worker-webhook で受信し、handlers/stripe/ 配下の 9 ハンドラに分岐する。

- invoice.paid
- invoice.payment_failed
- customer.subscription.created
- customer.subscription.updated
- customer.subscription.deleted
- customer.subscription.trial_will_end
- charge.refunded
- charge.dispute.created
- customer.tax_id.updated

詳細は [../02_基本設計/02_API設計.md](#) を正本とする。Stripe Webhook 受信は運営者側を正本とし、メイン側は IF #10 で受信する。

1.4.9 3.7 PCI-DSS スコープ外宣言

本サービスはクレジットカード情報を一切扱わず、Stripe Elements でクライアント直接送信。SAQ A の自己評価で十分。

1.4.10 3.8 80% 警告 / 100% / 125% アラート

question_logs を月次累計で集計し、無料枠の 80% / 100% / 125% 到達時に `usage.warning.80 / usage.limit.100 / usage.limit.125` を `retention_class=billing` で監査記録 + オーナー通知。125% 超過時は受信レートを 1/2 に制限する。

1.4.11 3.9 関連する横断設計

- 通知: `BILLING_INVOICE_ISSUED / usage.warning.80` 等の通知配信は [DD07_通知ロジック.md](#)
- 監査: `billing.* / usage.*` は `retention_class=billing` で 7 年保持 ([DD10_監査ログ書込・完全性検証.md](#))
- cron: 月次集計・月次確定の詳細は [DD14_バッチ・非同期処理.md](#)

1.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
課金・請求設計	../02_基本設計/10_課金・請求設計.md
API 設計	../02_基本設計/02_API 設計.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md
関連 DD	DD07_通知ロジック.md / DD10_監査ログ書込・完全性検証.md / DD14_バッチ・非同期処理.md

1.6 5. テスト観点

AC ID	テスト ID	テスト方式	テストファイル
AC-020	e2e-usage-001	E2E	apps/admin/e2e/usage/dashboard.spec.ts
AC-040	it-stripe-events-001	Integration	workers/webhook/test/integration/stripe.test.ts

1.6.1 5.1 その他観点

観点	内容
単体	calculateInvoiceAmount 各プラン × 各使用量パターン
結合	月次集計 → 月次確定 → IF #10 送信 → 通知配信
異常系	月次確定の幂等キー重複（同月再実行）でも二重生成しない
境界値	80% 警告 / 100% / 125% 拒否のぴったり境界
性能	月次集計 200 契約で 60 分以内完了
7 年保持	<code>billing.* usage.* action</code> コードの <code>retention_class=billing</code> 確認

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済